Can an Agent's Passing Test Result Be Reproduced? A Feasibility Study of Attempt-Time Dependency Capture

Anonymous submission

***Abstract*—An agent's statement "the tests pass" is true in the environment it ran in; whether a reader can reproduce it is a separate question. On 40 randomly drawn agent-authored pull-request tasks (a pre-registered study; one model, Python only), 22 runs were valid (the agent's target tests pass and fail without its changes). Replaying the agent's `pip freeze`, captured when it ran its tests, in a clean sandbox reproduced 20 of 22 passing results (91%, 95% CI 72–97), against 12 (55%, 35–73) for a zero-config reconstruction from the repository; capture alone reproduced 8, reconstruction alone 0 (paired difference 36 points, 95% interval 15–55; exact sign test $p = 0.008$). Of the reconstruction's 10 misses, 7 were UNKNOWN because it found no dependencies to install, 1 produced no result, 1 was a credential skip and 1 a real failure. The comparison is weak by design: reconstruction is a heuristic and capture replays the environment where the tests already passed, so we read it as an upper bound on replayability, not a verification method. The pre-registered rule is met at the minimum power boundary (22 valid runs, floor 20), in the fourth capture study. Two follow-ups registered after the study and before they were run point the same way: against a stronger reconstruction (CI workflows, Dockerfiles and extra requirements files, 18 re-runnable runs) capture reproduced 16 and the baseline 9, with no run reproduced by the baseline alone; and the 16 re-testable captures re-ran unchanged 8 days later. No case shows replay changing a verdict. We report the limits and an instrument defect that only real repositories exposed.**

I. Introduction

Coding agents report that tests pass. Whether anyone else can reproduce that depends on state a reader cannot see: installed packages, environment variables, generated files. Prior work measures whether agents can build environments (SetupBench [1]), whether an agent's self-reported capabilities survive hidden probes (ResearchEnvBench [2]), and whether an agent's declared dependencies match a runtime trace [3]. Tools that bind an agent's claim to a recorded run exist (Section VII). We claim no new verification mechanism. The question is empirical and narrow: *on real agent-authored pull requests, does recording the agent's Python dependency state when it runs its tests make its passing result reproducible more often than reconstructing the environment from the repository?* One possible gap, not verified against all prior work, is this measurement on a frozen patch with several replay contexts compared.

II. Method

Contexts. Each task's agent patch is frozen. The same scoped target command (the tests the agent claims) is replayed in a clean sandbox (macOS `sandbox-exec` or Linux bubblewrap; network off for tests) under: (i) *reconstruction*, a zero-config ladder (`pip install .[test|dev|...]`, a fixed list of requirements files, dependency groups, plus pytest); (ii) a *recipe* the agent writes, validated against a closed vocabulary; (iii) *capture*: the agent's `pip freeze` after its run, with `pip` dropped, `setuptools` and `wheel` kept, editable and direct-URL entries dropped and the project's own distribution removed. No language model decides a result; a result is pass, fail or UNKNOWN, and UNKNOWN is never converted into a pass.

Sample and rule (pre-registered). 40 tasks drawn at random from 747 structural candidates (agent-authored PRs with Python tests, a fixed filter on a public dataset of agent PRs) unused in earlier stages, at most two per repository (31 repositories). Claude Code ran each in an empty environment. A run is *valid* if the agent's target tests pass in its own environment and fail on the baseline without its changes. Rule fixed before the run: supported if capture $\geq 70\%$ of valid runs and the net paired lead over reconstruction ≥ 15 points; underpowered below 20 valid runs. The 15-point threshold was set after earlier stages and before any data of this study. The run stopped once at 21 attempted tasks when the disk filled; the stop was not data-driven, but the interim numbers (capture 11/13, reconstruction 6/13 on valid runs) were seen before the registered plan resumed, and the 19 later tasks added 9 valid runs (capture 9, reconstruction 6). Funnel: 40 attempted, 1 setup failure, 39 ran, 30 with the agent's tests passing, 8 trivial (tests pass without the change), **22 valid** (19 repositories; 21 from PRs of one agent family).

III. Instrument

Verdict classes are CONTRADICTED, SUPPORTED_BY_PRIOR_EVIDENCE, NOT_CONTRADICTED and UNKNOWN; this study uses only whether a replay of the target tests passes. Replay fails closed: it refuses when no environment can be built, refuses setup that installs the project's own name or runs a symlinked or file-modifying prepare step, requires that the command execute the changed tests, and stops an install that exceeds a size, disk or time limit (reporting that the environment could not be built). Environment setup uses live package indexes, so determinism was measured same day only. The agent's recipe is an untrusted proposal; the agent's own freeze is a record, not evidence of correctness.

IV. Results

TABLE I
PAIRED OUTCOME ON THE 22 VALID RUNS.

outcome	runs
both reproduced	12
capture only	8
reconstruction only	0
neither	2

TABLE II
REPRODUCED RESULTS BY CONTEXT (WILSON 95% CI).

context	valid (22)	CI	all 40
reconstruction	12 (55%)	35–73	17
capture	**20 (91%)**	72–97	26
agent recipe (scoped)	17 (77%)	57–90	23

The rule is met (capture 91% $\geq$ 70%; lead 36 $\geq$ 15 points), at the minimum power boundary; the Newcombe interval of the paired difference is 15–55, so its lower bound equals the threshold. Counts over all 40 attempted tasks point the same way (26 against 17).

Why reconstruction misses. Seven UNKNOWN because the ladder found no dependencies to install, one with no ladder result, one credential skip (also missed by capture), one real failure (3 failing tests). The headline gap is mostly a statement about a weak zero-config comparator. **The two capture misses:** a repository whose importable package is not at its root after capture removed the project's own distribution (a design limit), and a suite that skips without an API key (a credential outside the captured packages).

TABLE III
THE 10 RECONSTRUCTION MISSES AMONG THE 22 VALID RUNS, AND WHAT CAPTURE DID ON THE SAME RUN.

reconstruction outcome	runs	capture
no dependencies found to install (UNKNOWN)	7	6 pass, 1 miss
no ladder result	1	pass
credential skip (API key unset)	1	miss
real test failure (3 failing tests)	1	pass

Post-hoc checks (descriptive, not part of the rule). Requiring one executed passing test changes one reconstruction result (a "pass" with only skipped tests): 11/22, net lead 41 points. The 8 capture-only runs lie in 8 different repositories; a repository-level bootstrap (19 repositories) gives a net-lead interval of 18–65 points. Among passing results, reconstruction has 1 of 12 with zero passed tests and capture 1 of 20 with some skipped tests beside passes. The replay puts the repository root and `src` on the import path in every arm; the effect was *not* measured (2 of the 20 capture passes touch files under `src/`).

Follow-up 1: a stronger reconstruction. The main objection to the comparison is the weak zero-config ladder. After B6 and before running it, we registered a stronger baseline derived only from files at the base commit: the ladder's own steps plus every `pip install` line in the CI workflows (also `tox.ini` dependencies and uv/poetry sync translated to pip), `pip install` lines in Dockerfiles, and the other requirements files, each validated by the tool's recipe validator; the test command is the same scoped command as in every context. The extractor was frozen first and no B6 outcome was used to design it, but the author had seen the B6 results. Four of the 22 valid runs cannot be rebuilt from the saved patch (the agent created untracked files the patch lacks) and are excluded, not counted as failures. On the remaining 18 runs the zero-config ladder reproduced 10, the strong baseline 9 and capture 16 (89%); capture alone 7, the baseline alone 0 (net lead 39 points; registered reading: the advantage survives a stronger comparator). Six baseline misses are missing dependencies that none of the files read supplies (not individually checked by hand), one is the credential skip, one a real failure, and in one run the extra CI steps broke a run the ladder passed.

Follow-up 2: cross-day stability. Eight days after the original replays, 16 of the 16 originally passing captures that could be re-tested still passed. The registered rule (at least 18 of 20) could not be met because only 16 were testable; we report that rather than repair the rule. Stability over weeks, on another platform, and for the four untestable runs is not shown.

V. Earlier exploratory studies of agent-declared recipes

Two earlier registered studies used 32 real pull requests from 22 repositories, chosen because zero-config reconstruction already reproduced the PR's own tests, so they could not show that anything beats reconstruction. In an empty environment, reconstruction reproduced 31/32, the agent's recipe 24/30 as registered, and its exact `pip freeze` 31/32; the registered hypothesis that recipes beat reconstruction was not supported (recipe-only 0 runs, reconstruction-only 7). With 17 common packages preinstalled in the agent's environment, reconstruction stayed at 31/32 while the scoped recipe fell to 15/32; 11 of its 17 failures were an omitted `pytest`, and a post-hoc change that supplies the test runner recovered 5 of 10 re-replays. Reading: on tasks filtered to be reconstructable, agent-declared recipes were the fragile context (omitted runners and packages, rejected vocabulary), and neither cohort could show whether recipes help where hidden state matters. In the present study the recipe arm (17/22) is reported but not part of the rule; five of its runs had a recipe rejected before replay.

VI. Development history and an instrument defect

Capture was first tested in three earlier studies on tasks selected because reconstruction had failed: 5/11 against 0/11 (inconclusive against the registered bar), 5/8 against 2/8 (below the registered minimum of 10 valid runs), 9/18 against 3/18 (inconclusive). After the third, five capture failures reported a "missing dependency" although the package was pinned

TABLE IV
CAPTURE STAGES (VALID RUNS; CAPTURE VS. RECONSTRUCTION).

stage	status	result
1	registered; tasks chosen where reconstruction failed	5/11 vs 0/11
2	registered; fresh tasks; below 10 valid	5/8 vs 2/8
3	registered; fresh tasks; inconclusive	9/18 vs 3/18
4 (this study)	registered; random sample; frozen protocol	20/22 vs 12/22

in the captured list. The cause was the tool's own 700 MB environment-size guard killing the install, after which tests ran in a partial environment; the message now says the environment could not be built. Raising the limit to 3 GB recovered 4 of the 5 (post hoc). A fixed resource limit that silently turns build failures into dependency verdicts is a general risk for replay-based verification. The capture design changed between stages, which is why this study used fresh tasks and a frozen protocol; readers should still weigh it as the fourth study.

VII. Related work

SetupBench [1] (93 tasks; success is a validation command run after the agent's last action; no check of the agent's completion claim) and ResearchEnvBench [2] (44 repositories; a hallucination metric compares self-reported success with hidden probes) establish auditing a self-report against an independent probe. Vangala and Malik [3] (3 agents, 4 languages, 50 tasks) audit declared dependencies against a runtime trace and call never-loaded declared packages overclaimed; they do not audit a passing-test claim. Open-source tools bind an agent's claim to a recorded run: `truth` [4] (a language model parses the claim, fixed rules decide; "tests pass" needs a recorded run; it never runs the command itself), ProofRun [5] (runs the check in the agent's environment, binds it to a code-state hash, reports stale on change), Proofrail [6] (checks claims against a commit range) and Trial [7] (a rules-file convention). As described in their READMEs (the authors' own descriptions), none re-executes the result in a separate clean environment and compares contexts; this is an absence claim from README text and our searches may have missed work. Patents on hashed test environments and environment fingerprints exist and were not reviewed by counsel.

VIII. Discussion

Three points follow, none beyond this sample. First, the result is about *replayability*: it says a reader can usually rerun the agent's tests in the agent's package state, not that the tests are meaningful or the claim true. Second, the large share of reconstruction misses that are UNKNOWN (7 of 10) is a reminder that a verifier should report UNKNOWN rather than a failure when it simply could not build the environment (we did not test whether this changes any reader's decision); the instrument defect in the previous section is the same lesson from the other side. Third, if a verifier wants its result to be reproducible, recording the package state at the moment the agent ran its tests is cheap (one `pip freeze`) and was sufficient here for Python packages; it is silent on system packages, services, generated files and secrets, which this study could not examine.

IX. Threats and limits

One model, one run per task, Python only, small repositories (many hobby projects; the two capture misses are the two with real infrastructure), valid runs conditioned on the agent's success (21 of 22 from one PR-author family). Capture comes from the claimant's own environment, so "independent" means independent of the sandbox, not of the claimant. All replays ran on one macOS host on one day against live package indexes (yanked or platform-specific wheels may break a pinned capture later). The target tests are the verified object, not the agent's sentence. No run shows replay changing a verdict or catching a false claim, and no deceptive agent was studied. Reproduction is not correctness. The strong baseline is one reasonable strong comparator (it does not read READMEs, install dependency groups, pick the interpreter version or provide system packages). A post-hoc rule in the tool that downgrades a contradiction to UNKNOWN when a change edits dependency manifests was evaluated in-sample.

X. Conclusion

For a random sample of real agent PRs, the agent's recorded Python packages reproduced 91% of valid passing results in a clean sandbox against 55% for a weak zero-config reconstruction. This supports only that the agent's environment is usually replayable and that the repository alone often does not say how. A first strong reconstruction baseline (CI, Dockerfile and extra requirements files) did not close the gap on 18 rerunnable runs. A full study would still need 50 or more valid runs across agents and PR-author families, replay weeks later and on Linux, a wider set of strong baselines, and a case where replay changes a decision.

Availability. Task identifiers, prompts, analysis scripts and the replay tool: anonymised artifact (URL given at submission). Model outputs and environment snapshots are not released. The CLI version and model ID per run were not recorded.

References

[1] A. Arora, J. Jang, R. Zilouchian Moghaddam, "SetupBench: Assessing software engineering agents' ability to bootstrap development environments," arXiv:2507.09063, 2025.

[2] Y. Wang *et al.*, "ResearchEnvBench: Benchmarking agents on environment synthesis for research code execution," arXiv:2603.06739, 2026.

[3] B. P. Vangala and T. Malik, "Code that works, environments that don't: Measuring environment reproducibility in AI-generated software," arXiv:2610.00425, 2026.

[4] "truth: a deterministic fact-checker for AI coding agents," GitHub repository blasrodri/truth, accessed 15 Oct. 2026.

[5] "ProofRun: a local verification receipt for AI coding agents," GitHub repository yebiguo/ProofRun, v0.3.0, accessed 15 Oct. 2026.

[6] "Proofrail," GitHub repository DrDeese/Proofrail (public alpha), accessed 15 Oct. 2026.

[7] "Trial," GitHub repository Da7-Tech/trial, accessed 15 Oct. 2026.